```
    <a asp-page="Edit" asp-route-id="@Model.Id"
        class="btn btn-primary">Edit</a>
}
```

This ensures that only users who will be able to execute the Edit.cshtml Razor Page can see the link to that page.

WARNING The `if` clause means that the Edit link will not be displayed unless the current user created the recipe, but you should never rely on client-side security alone. It's important to keep the server-side authorization check in your Edit.cshtml page handler to protect against any direct access attempts. Even if a malicious user circumvents your UI, the server-side authorization ensures that your application is secure.

With that final change, you've finished adding authorization to the recipe application. Anonymous users can browse the recipes created by others, but they must log in to create new recipes. Additionally, authenticated users can edit only the recipes that they created, and they won't see an Edit link for other people's recipes.

Authorization is a key aspect of most apps, so it's important to bear it in mind from an early point. Although it's possible to add authorization later, as you did with the recipe app, it's normally preferable to consider authorization sooner rather than later in the app's development.

In chapters 23 and 24 we focused on authentication and authorization for traditional web applications using Razor. In chapter 25 we'll look at API applications, how authentication works with tokens, and how to add authorization policies to minimal APIs.

Summary